Reeves Louis
36493

Recherche des meilleurs chemins en escalade



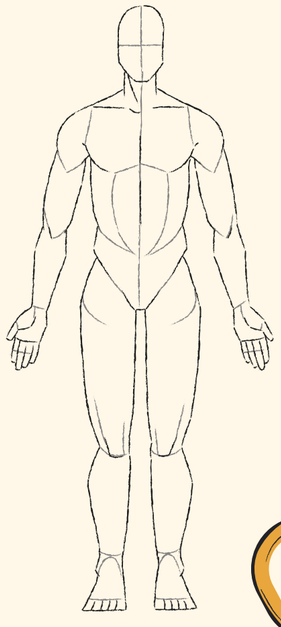


Analyse du mur

Facteurs artificiels



Facteurs humains



Conclusion

Programme Version 2

Coût de l'effort

Programme Version 1

Modélisation d'approche

Présentation

Comment déterminer les trajets les plus efficaces grâce à l'algorithmie ?



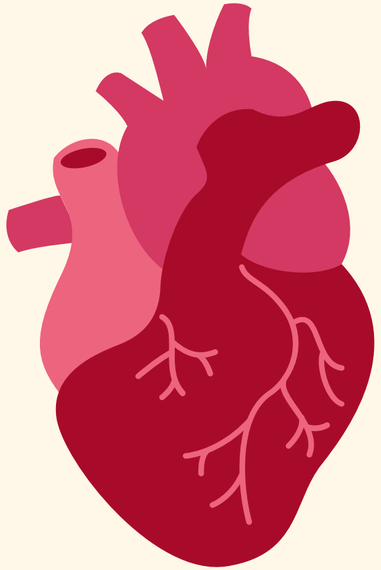
Planetgrimpe.com



fr.freepik.com

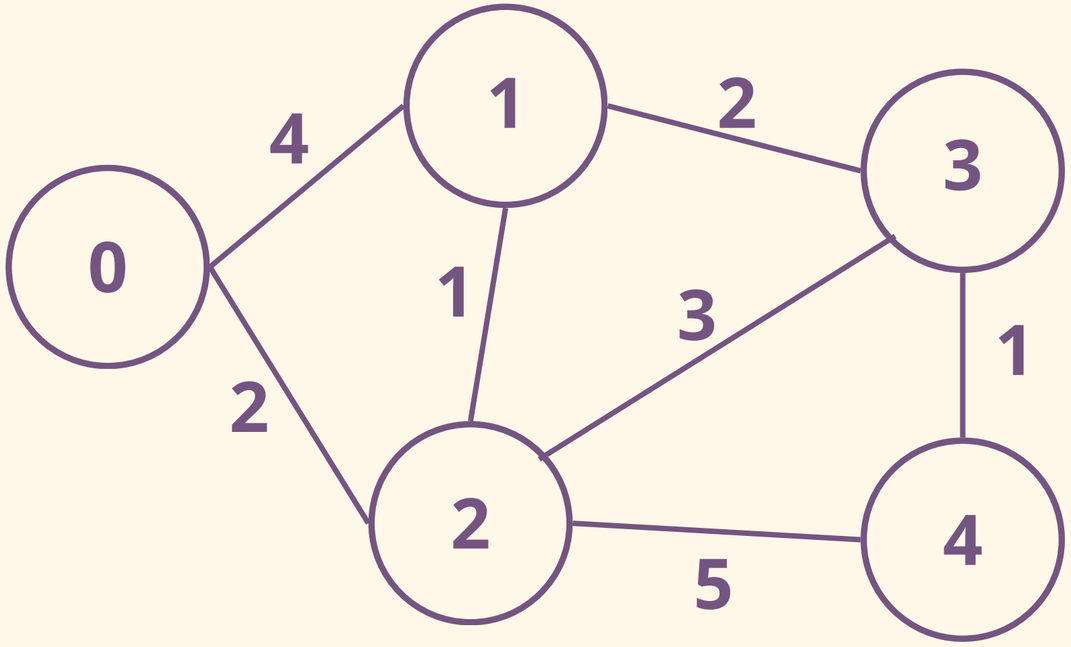
(A : temps de recherche réduite sous condition)*

Algorithme de Dijkstra



Entrée : Un graphe pondéré et le sommet de départ S_0

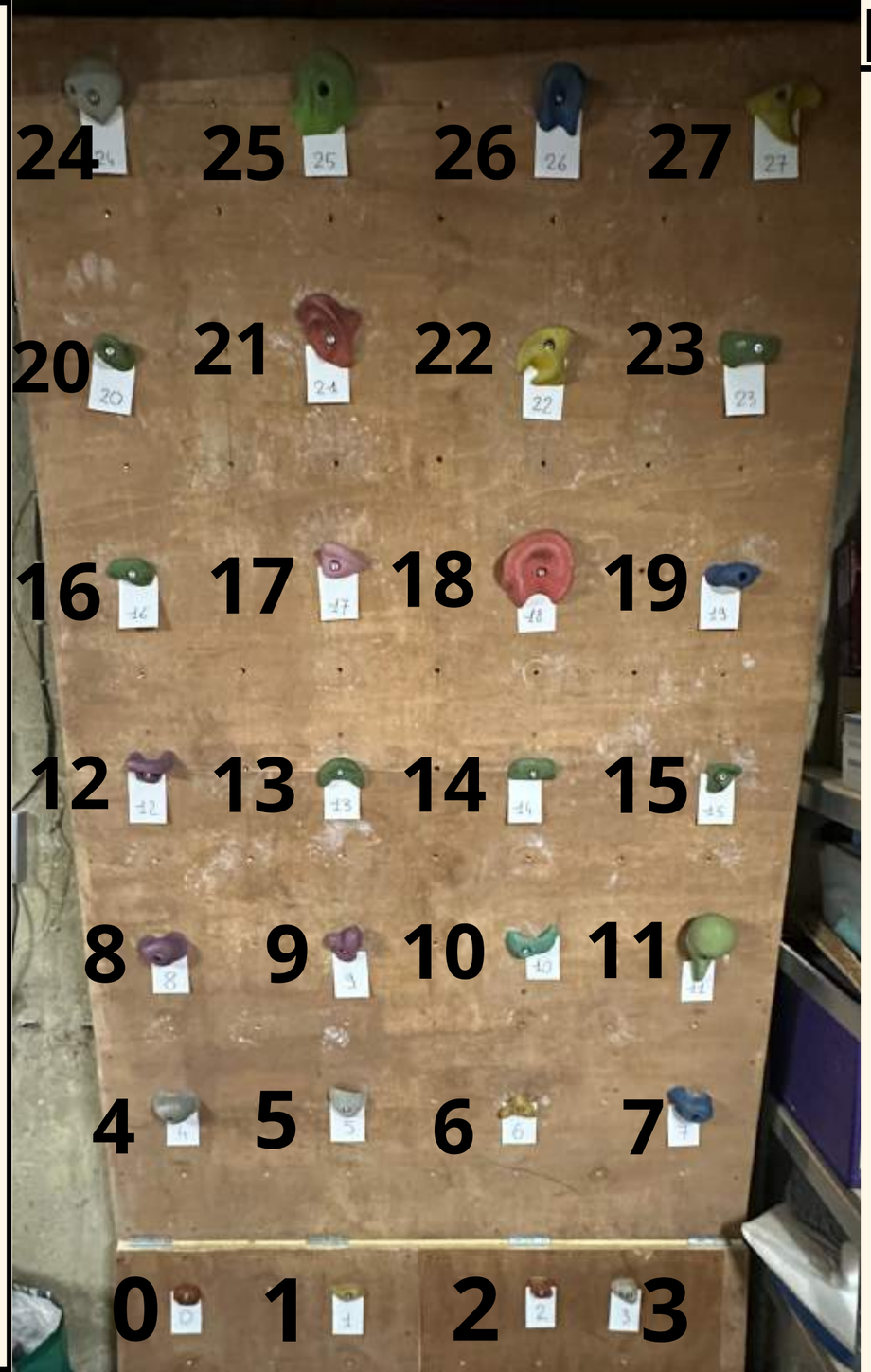
Sortie: La distance la plus courte entre S_0 et chaque sommet



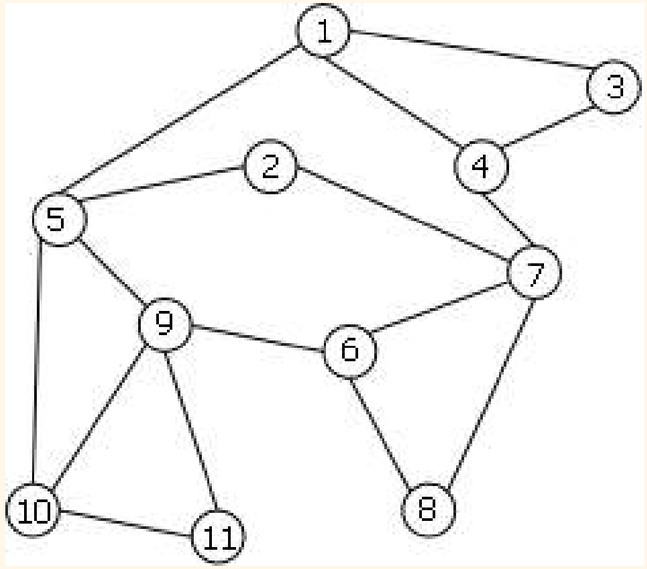
$$dist = [0,+\infty,+\infty,+\infty,+\infty]$$

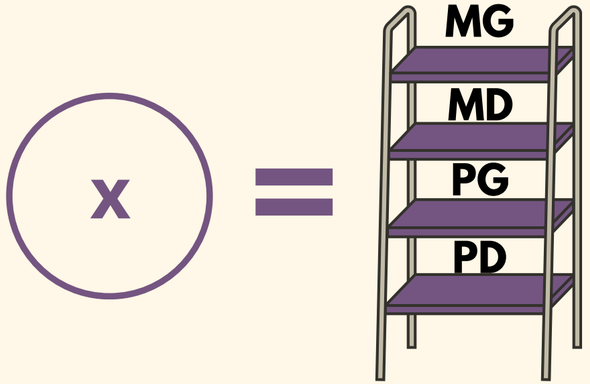
$$pred = [-1,-1,-1,-1,-1]$$

Conclusion
Programme Version 2
Coût de l'effort
Programme Version 1
Modélisation d'approche
Présentation



Modélisation d'approche





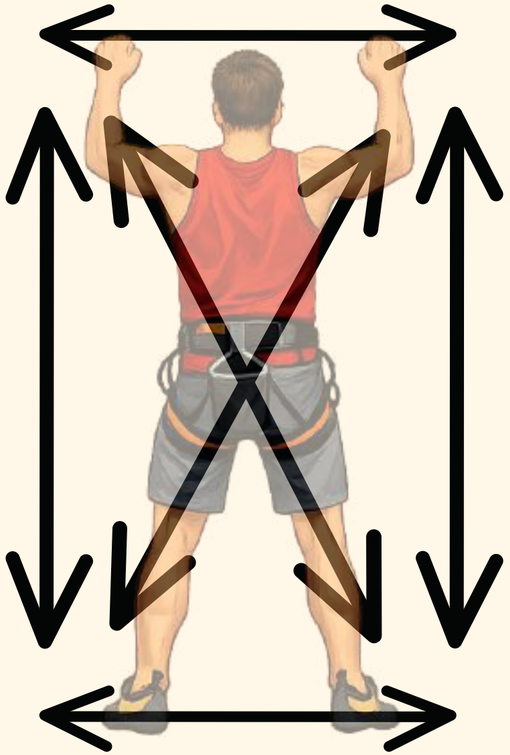


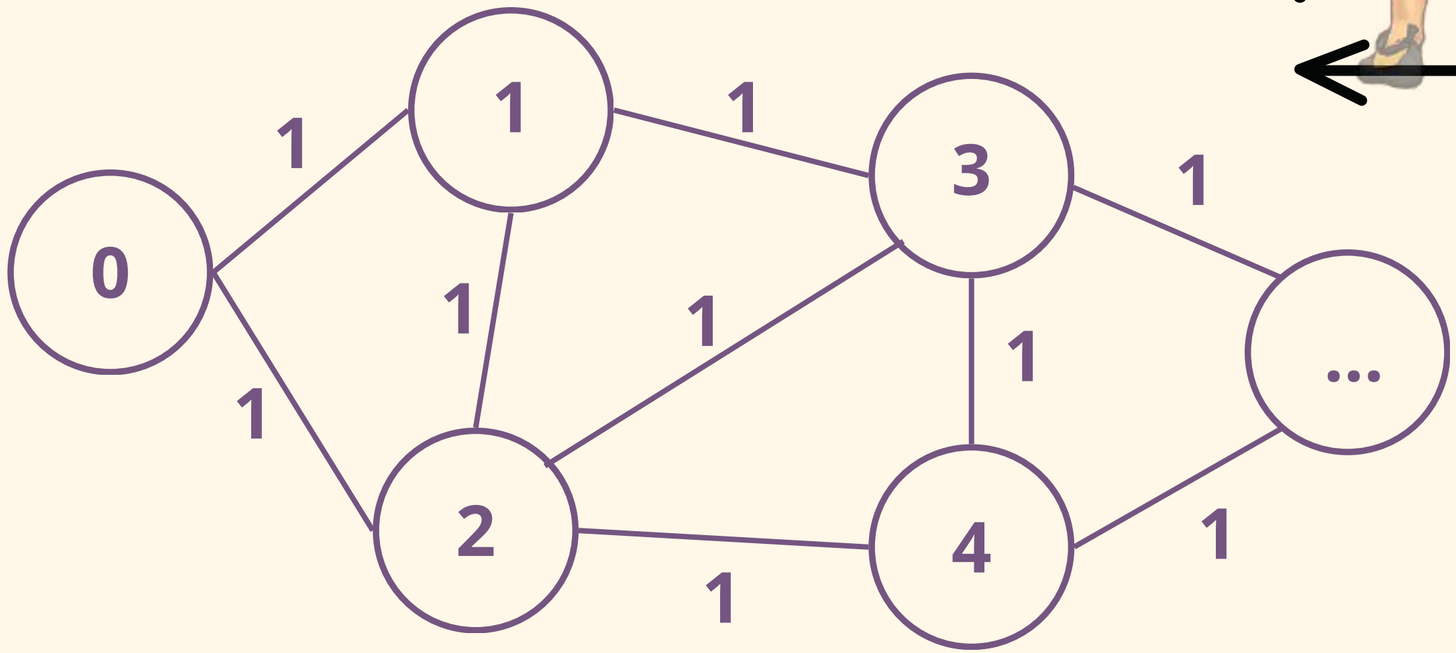
Conclusion
Programme Version 2
Coût de l'effort
Programme Version 1
Modélisation d'approche
Présentation

Modélisation d'approche

- un seul membre par prise
- mouvements considérés statiques
- déplacement d'un seul membre à la fois

Le grimpeur :





Conclusion

Programme Version 2

Coût de l'effort

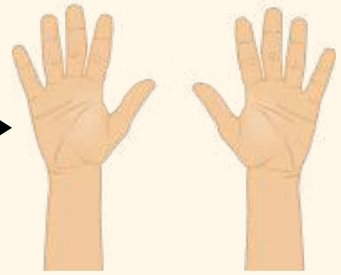
Programme Version 1



Modélisation d'approche

Présentation

Les limites de distance

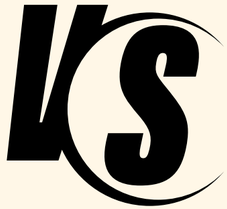


+



+ ...

Est_admissible
Annexe 2



Hypothèse en plus :
Mains > Pieds

La liste des prises



Prises_a_sommets
Annexe 3



Recherche de tous les états

Est_adjacent
Annexe 4



Recherche états avec 3 points communs

Conclusion

Programme Version 2

Coût de l'effort

Programme Version 1

Modélisation d'approche

Présentation





Conclusion

Programme Version 2

Coût de l'effort



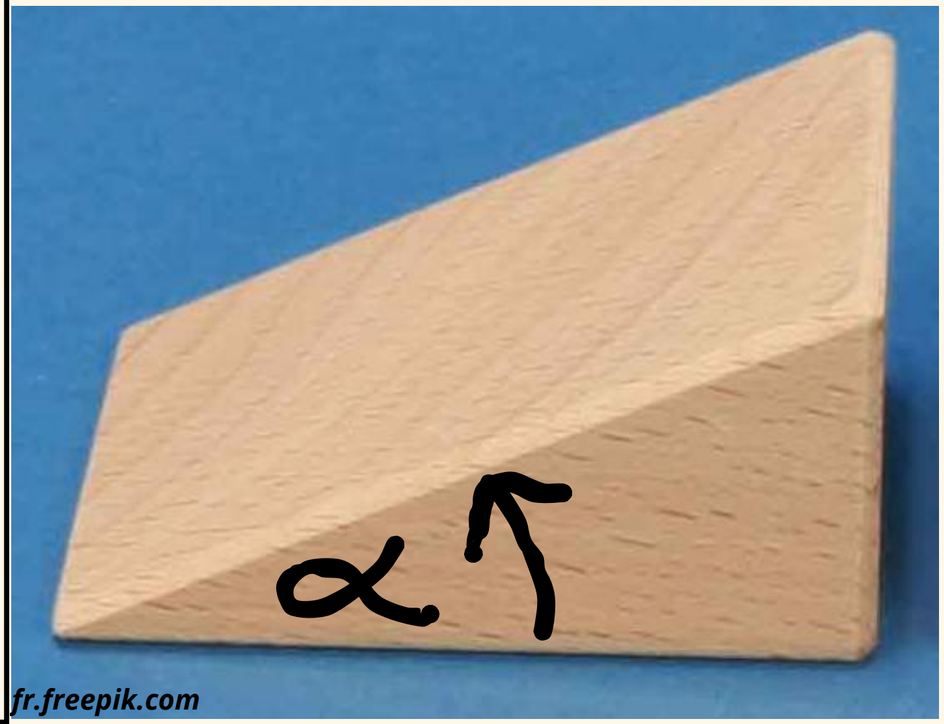
Programme Version 1

Modélisation d'approche

Présentation



Planetgrimpe.com



fr.freepik.com

Effort maximal applicable sur la prise ?

Conclusion

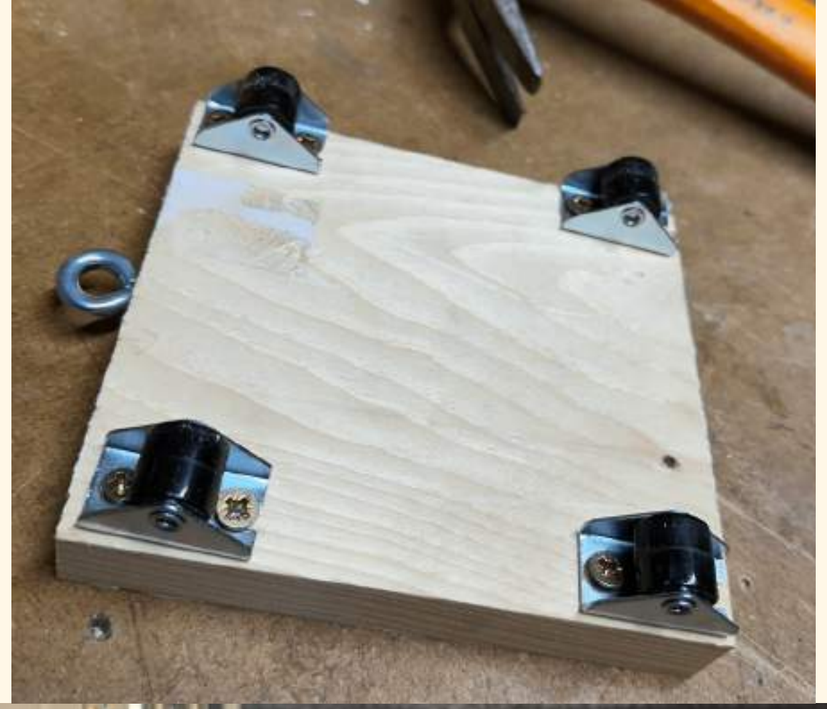
Programme Version 2

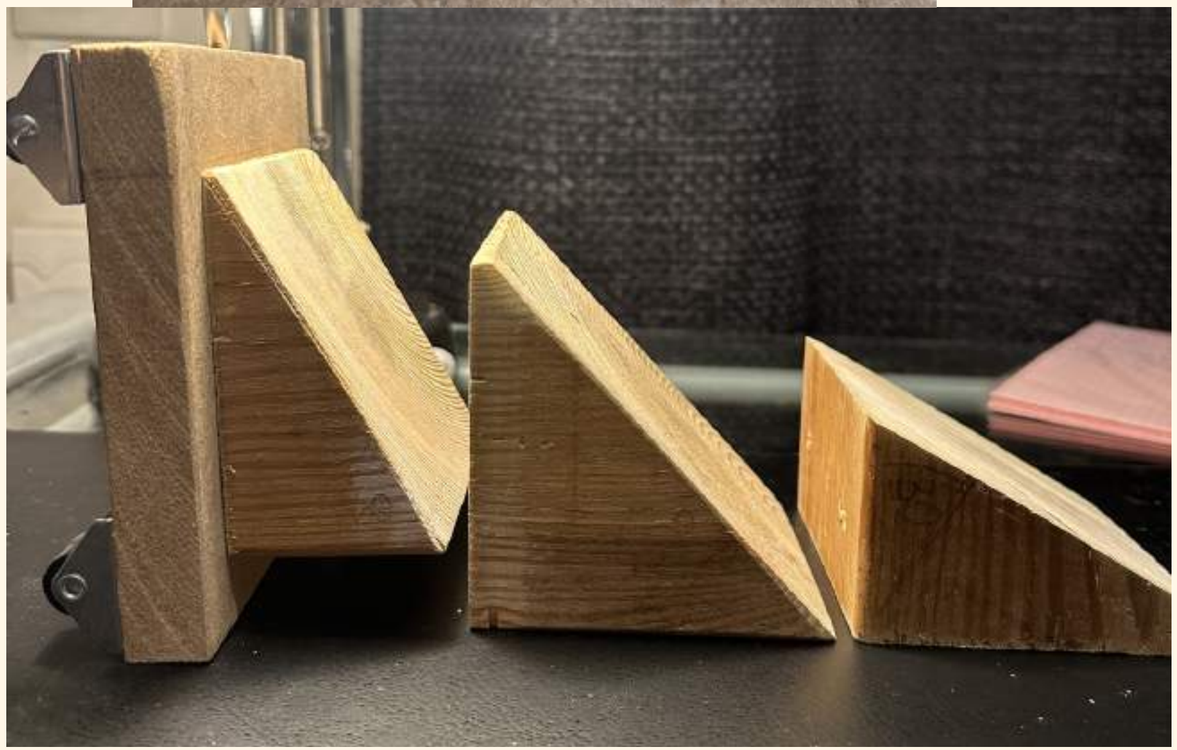
Coût de l’effort

Programme Version 1

Modélisation d’approche

Présentation







Conclusion

Programme Version 2

Coût de l’effort



Programme Version 1

Modélisation d’approche

Présentation





Conclusion

Programme Version 2

Coût de l'effort

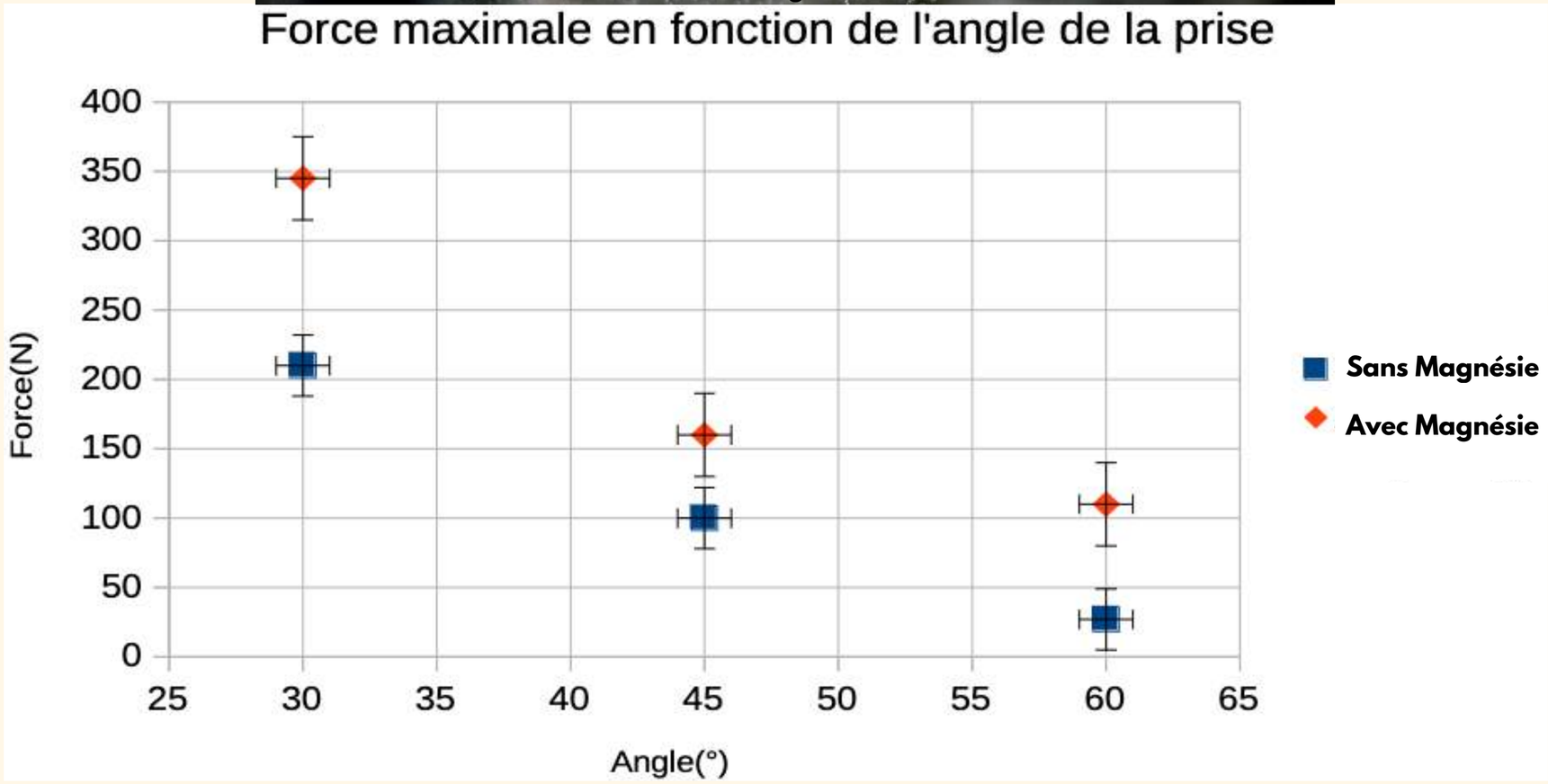


Programme Version 1

Modélisation d'approche

Présentation





Conclusion

Programme Version 2

Coût de l'effort

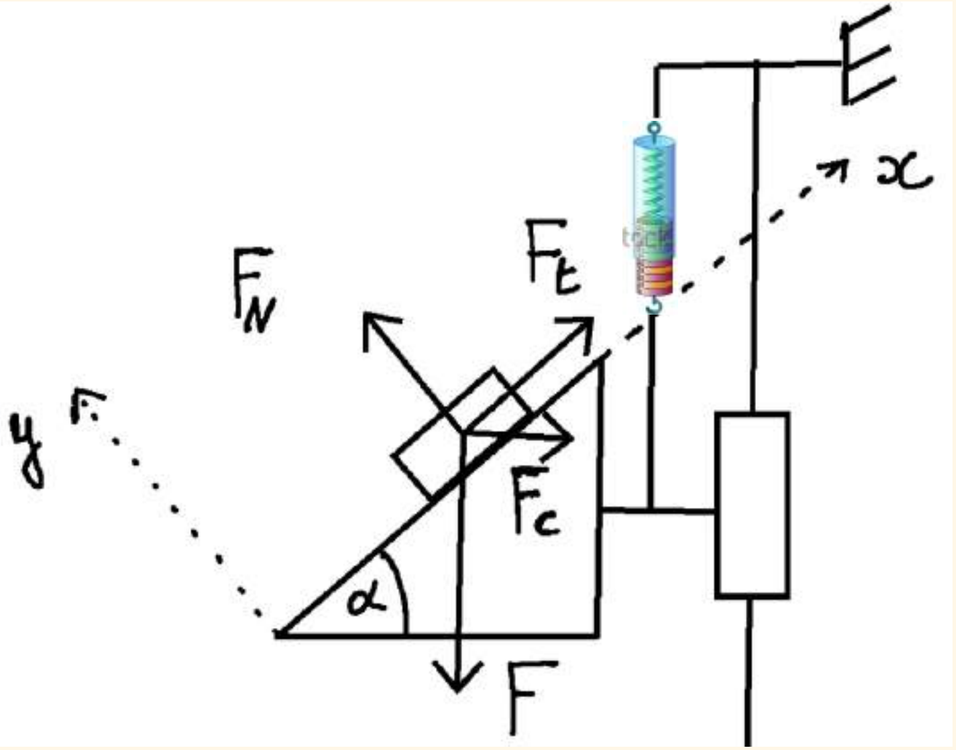


Programme Version 1

Modélisation d'approche

Présentation

Théorie



TRS sur le doigt :

Sur x : $-F.\sin(\alpha) + Fc.\cos(\alpha) + Ft = 0$

$\Rightarrow Ft = -Fc.\cos(\alpha) + F.\sin(\alpha)$

Sur y : $-F.\cos(\alpha) - Fc.\sin(\alpha) + Fn = 0$

$\Rightarrow Fn = Fc.\sin(\alpha) + F.\cos(\alpha)$

Avec : $Ft = f.Fn$

$$Fmax = \frac{Fc.(f.\sin(\alpha) + \cos(\alpha))}{\sin(\alpha) - f.\cos(\alpha)}$$

Conclusion

Programme Version 2

Coût de l'effort

Programme Version 1

Modélisation d'approche

Présentation

Obtenir Fc :

$$Fc = (35{,}63 \pm 1{,}18) N$$





Conclusion

Programme Version 2

Coût de l'effort



Programme Version 1

Modélisation d'approche

Présentation

$$\Rightarrow f = \frac{Fmax.\sin(\alpha) - Fc.\cos(\alpha)}{Fc.\sin(\alpha) + Fmax.\cos(\alpha)}$$

Sans magnésie : $f = 0,35 \pm 0,02$



Avec magnésie : $f = 0,5 \pm 0,03$



Conclusion

Programme Version 2

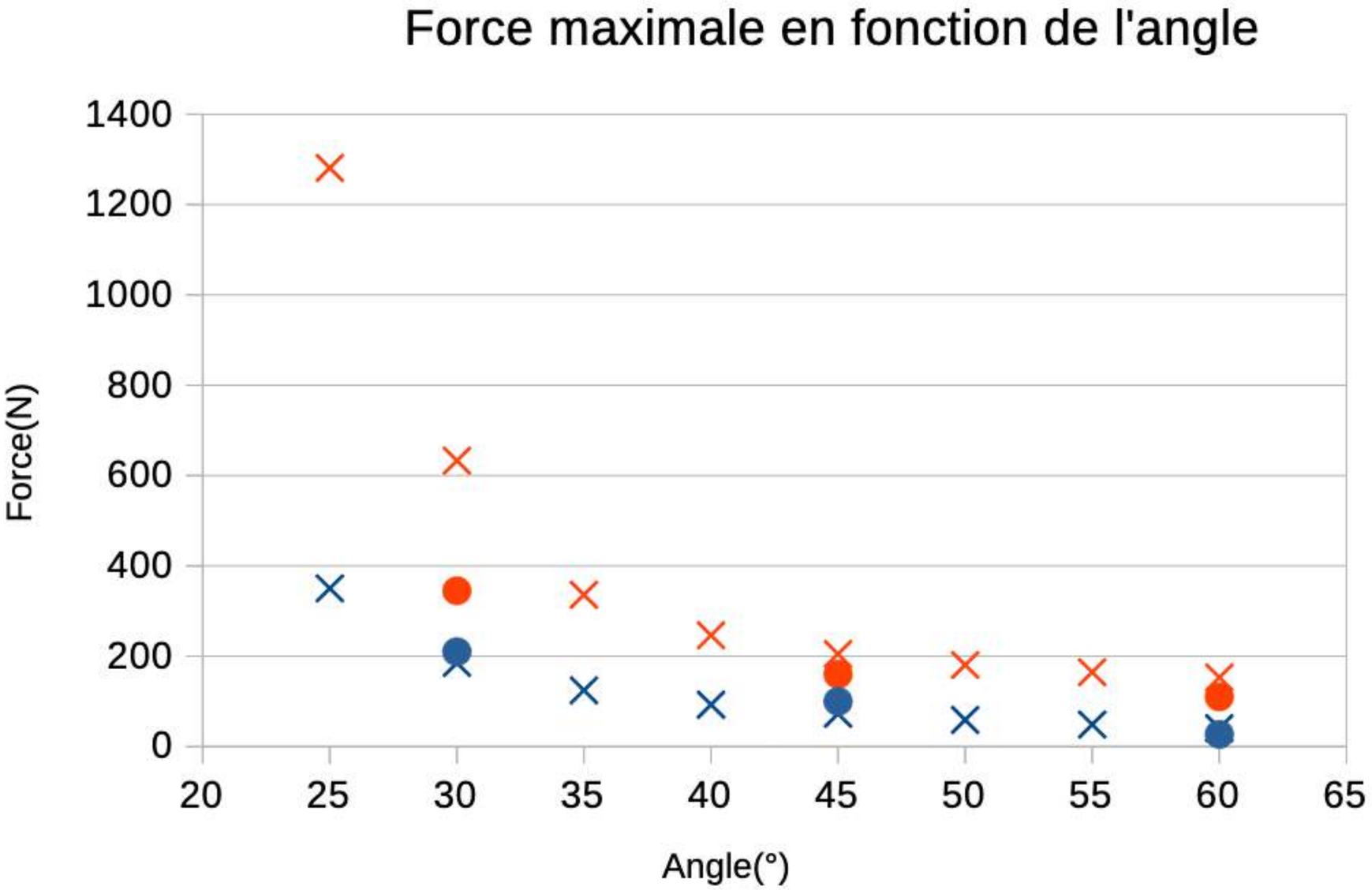
Coût de l'effort



Programme Version 1

Modélisation d'approche

Présentation



Conclusion

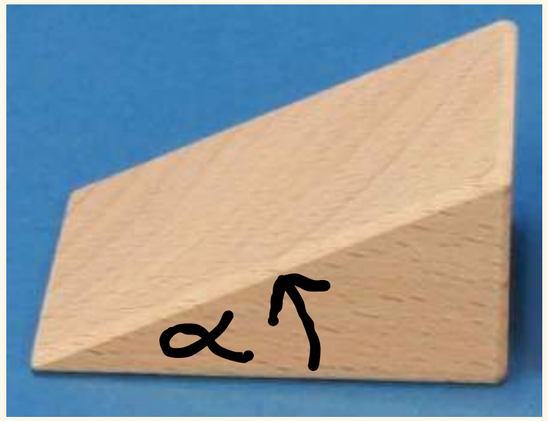
Programme Version 2

Coût de l'effort

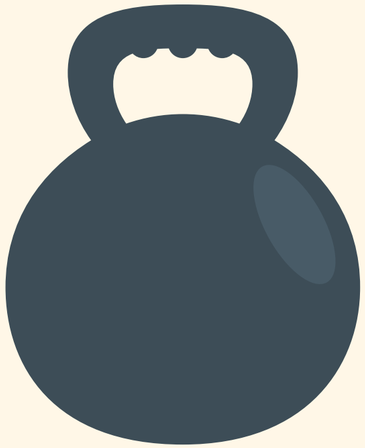
Programme Version 1

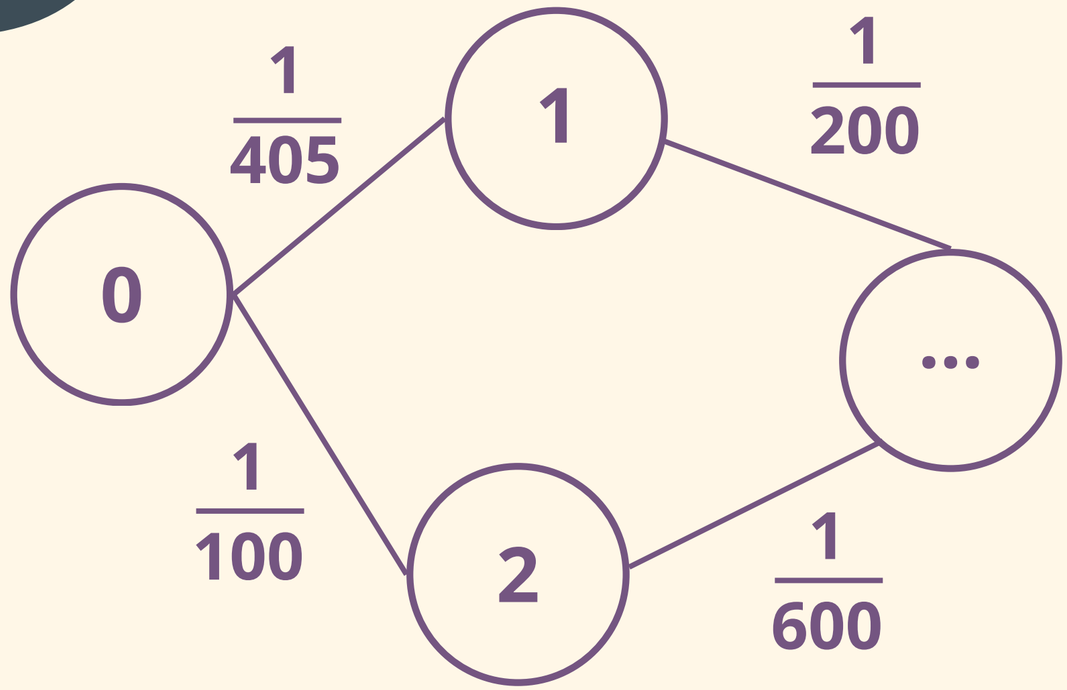
Modélisation d'approche

Présentation

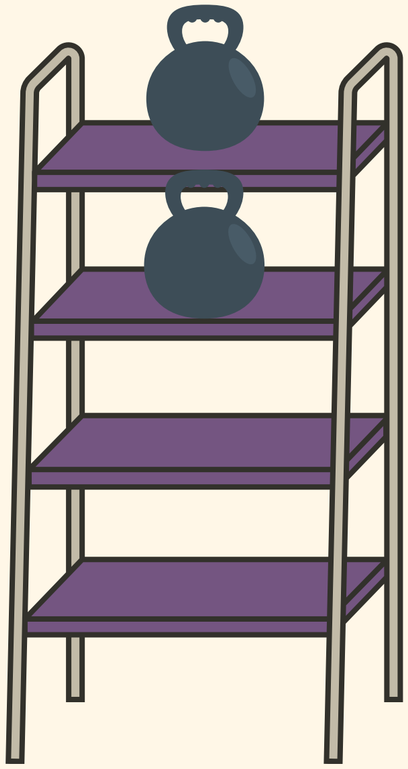


f ⟹





X =



MG

MD

PG

PD

Conclusion

Programme Version 2

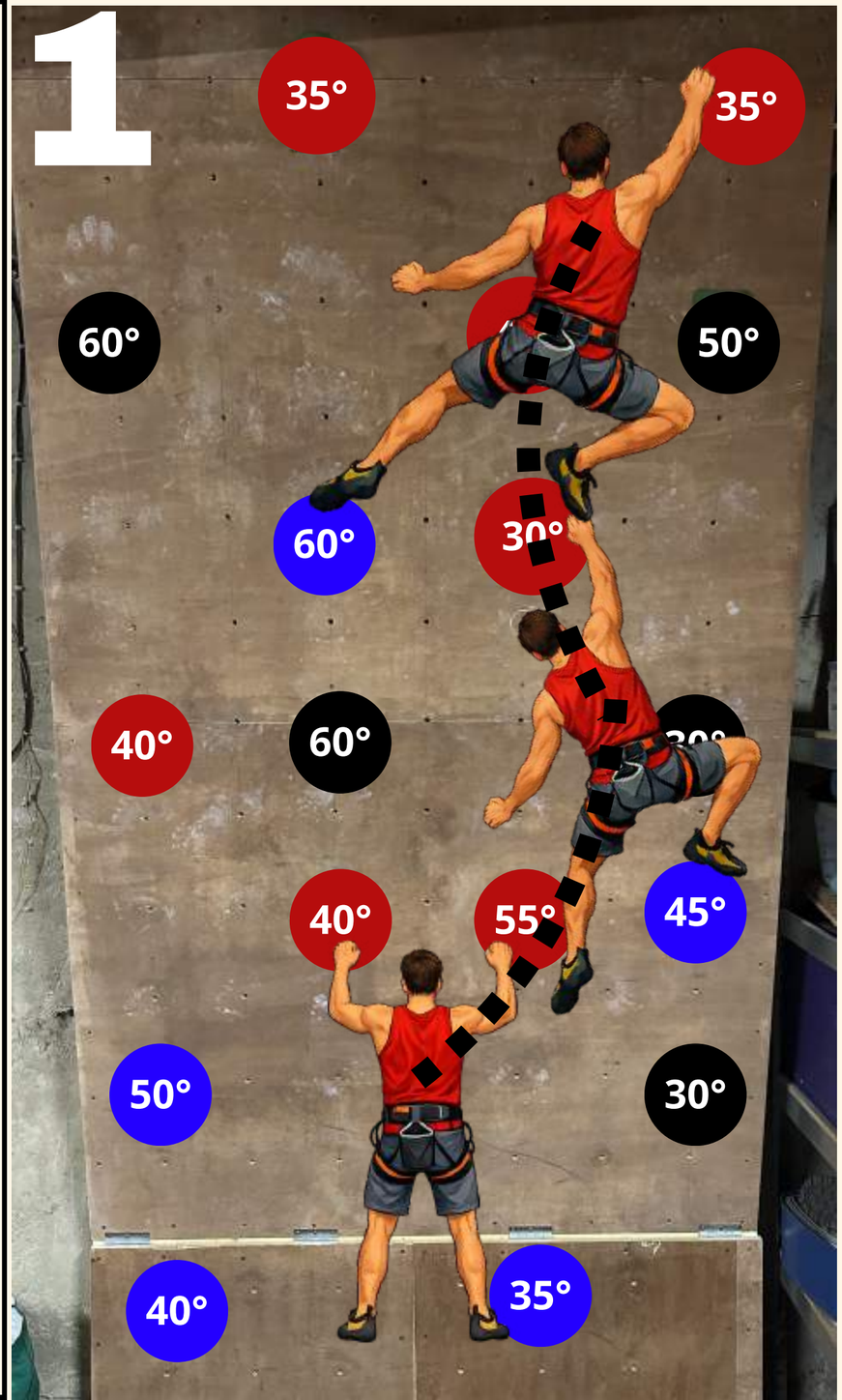


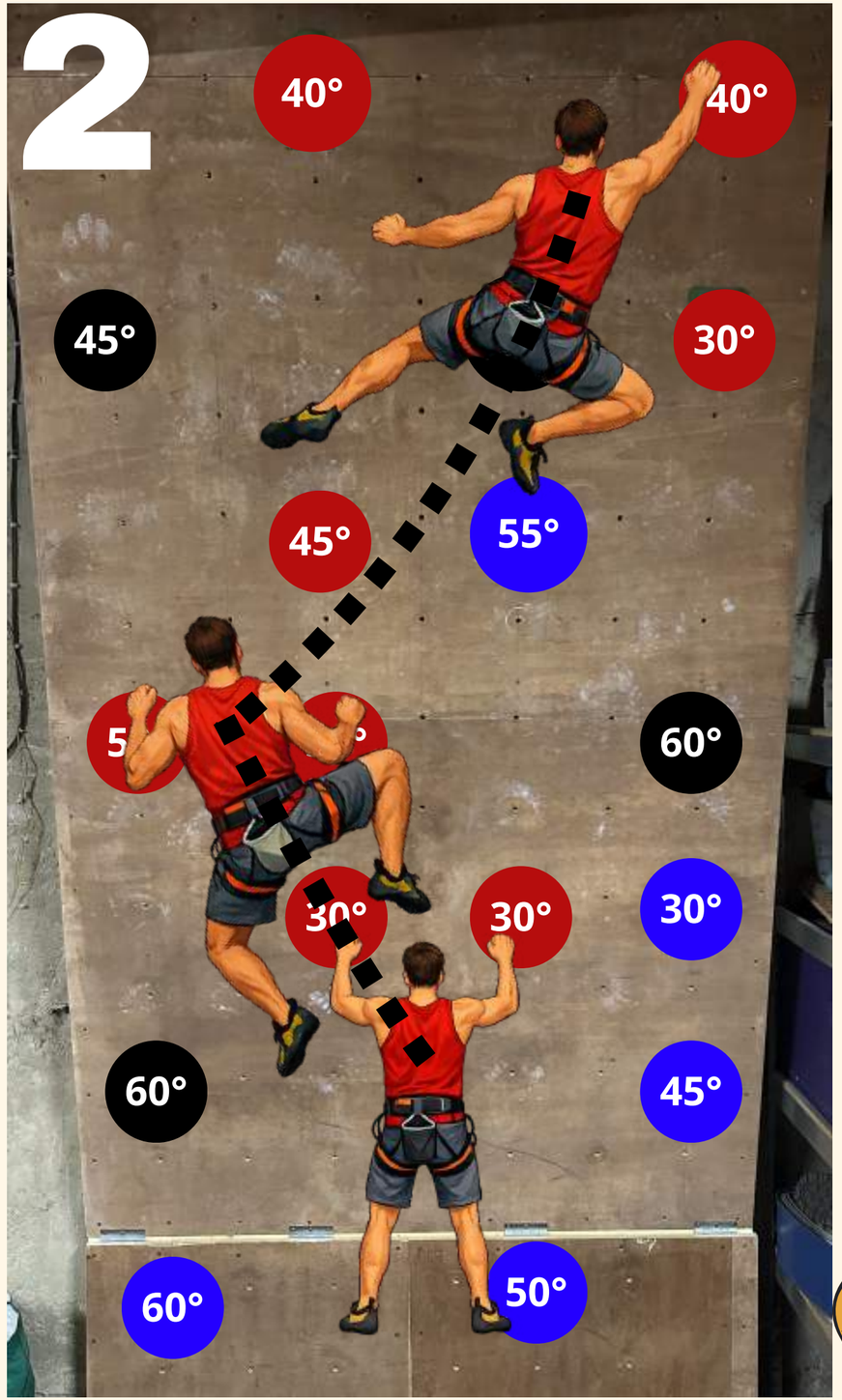
Coût de l'effort

Programme Version 1

Modélisation d'approche

Présentation





Mains

Pieds

Conclusion

Programme Version 2

Coût de l'effort

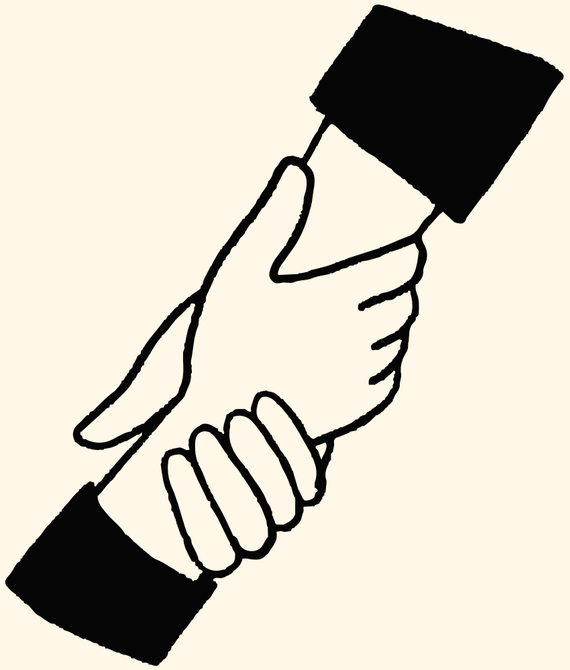
Programme Version 1

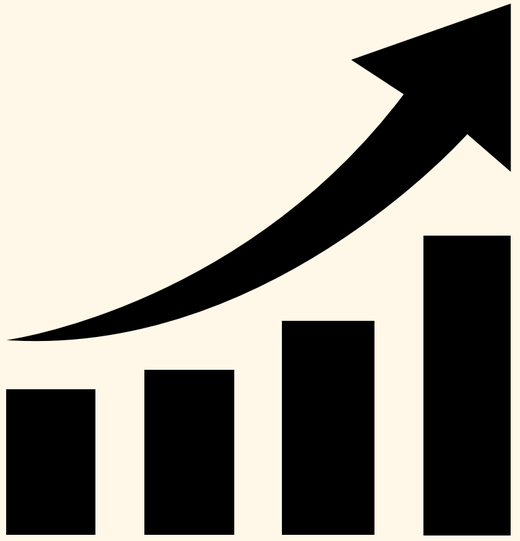
Modélisation d'approche

Présentation



fr.freepik.com







Planetgrimpe.com

ANNEXE 1 :

```
def distance(t1,t2):
    """

    Arguments
    ----------
    t1 : triplet  sous forme (x,y,angle)
    t2 : triplet  sous forme (x,y,angle)

    Sortie
    -------
    Flottant

    """
    (x1,y1),(x2,y2) = t1[:2],t2[:2]
    return sqrt((x2-x1)**2+(y2-y1)**2)
```

ANNEXE 2 :

```
MainG_MainD = (0.4,1.2)
MainG_PiedG = (0.79,1.6)
MainG_PiedD = (0.89,1.8)
MainD_PiedG = (0.89,1.8)
MainD_PiedD = (0.79,1.6)
PiedG_PiedD = (0.4,1.5)
```

```
def est_admissible(e):
    """
    Argument
    ----------
    e : Liste de longueur 4, de triplets;
    c'est un état qu'il faut vérifier

    Sortie
    -------
    Booléen
    """
    MG,MD,PG,PD = e

    # tests de distances (dmin, dmax)
    tests = [(MG,MD,MainG_MainD),
             (MG,PG,MainG_PiedG),
             (MG,PD,MainG_PiedD),
             (MD,PG,MainD_PiedG),
             (MD,PD,MainD_PiedD),
             (PG,PD,PiedG_PiedD)]
    for a,b,(d_min,d_max) in tests:
        d = distance(a,b)
        if not (d_min <= d <= d_max):#si conditions distances non remplies
            return False

    # contrainte "mains au-dessus des pieds", en se rappelant que MG,MD... sont des triplets(x,y,angle)
    if not (MG[1] > PG[1] and MG[1] > PD[1] and MD[1] > PG[1] and MD[1] > PD[1]):
        return False
    return True
```

ANNEXE 3 :

```
def prises_a_sommets(P):
    """
    Argument
    ----------
    P : Une liste de triplets, de taille égale au nombre de prises

    Sortie
    -------
    Une liste de liste, c'est les sommets possibles avec les prises
    """
    n = len(P)
    sommets_possibles = []
    for mg in range(n):
        for md in range(n):
            for pg in range(n):
                for pd in range(n):
                    candidat=[P[mg],P[md],P[pg],P[pd]]
                    if est_admissible(candidat):
                        sommets_possibles.append(candidat)
    return sommets_possibles
```

ANNEXE 4 :

```
def est_adjacent(s,L):
    """
    Arguments
    ----------
    s : liste de taille 4 composée de triplets, c'est un sommet
    L : liste de listes de triplets, c'est la liste de tous les sommets possibles

    Sortie
    -------
    Liste d'indices, c'est la liste de l'indice des sommets adjacents à s

    """
    n = len(L)
    adjacents_de_s = []
    for i in range(n):
        l = L[i]
        prises_communes = 0
        if s[0] == l[0]:
            prises_communes+=1
        if s[1] == l[1]:
            prises_communes+=1
        if s[2] == l[2]:
            prises_communes+=1
        if s[3] == l[3]:
            prises_communes+=1
        if prises_communes == 3: #si le sommet l a 3 "points communs" avec s
            adjacents_de_s.append(i) #alors on ajoute l'indice de ce sommet
    return adjacents_de_s
```

ANNEXE 5 :

```
119     def liste_de_listes_adjacence(L):
120         """
121         Argument
122         ----------
123         L : Liste de tous les sommets possibles
124 
125         Sortie
126         -------
127         liste_de_listes_adjacences : Liste de tous les adjacents de tous les sommets
128 
129         """
130         liste_de_listes_adjacences = []
131         n = len(L)
132         for s in range(n):
133             adj = est_adjacent(L[s], L)
134             liste_de_listes_adjacences.append(adj)
135         return liste_de_listes_adjacences
```

ANNEXE 6 :

```
def coord_a_indice(L,P):
    """

    Arguments
    ----------
    L : liste des sommets possibles avec les prises en coordonnées
    P : liste de la position des prises

    Sortie
    -------
    L_indices : liste des sommets possibles avec les prises en indices

    """
    L_indices = []
    for k in L:
        mg,md,pg,pd = k
        K = []
        for p in range(len(P)):
            if P[p] == mg:
                K.append(p)
        for p in range(len(P)):
            if P[p] == md:
                K.append(p)
        for p in range(len(P)):
            if P[p] == pg:
                K.append(p)
        for p in range(len(P)):
            if P[p] == pd:
                K.append(p)
        L_indices.append(K)
    return L_indices
```

ANNEXE 7 :

```python
from math import cos,sin,pi

def Fmax(prise):
    """
    Argument
    ----------
    prise : un triplet (x,y,angle)

    Sortie
    -------
    Un entier, la force maximale applicable sur la prise
    """
    x,y,angle = prise
    Fc = 35
    f = 0.5
    return(Fc*(f*sin(angle*pi/180)+cos(angle*pi/180))/(sin(angle*pi/180)-f*cos(angle*pi/180)))

def cout(s):
    """
    Arguments
    ----------
    s : sommet pour lequel on veut le coût de la position

    Sortie
    -------
    Un float, le coût du mouvement
    """
    mg,md,pg,pd = s
    return (round(1/Fmax(mg),3)+round(1/Fmax(md),3))
```

ANNEXE 8 :

```python
from math import inf

def dijkstra_V2(G,s0=0):
    '''
    Arguments
    ----------
    G : graphe stocké par liste des listes d'adjacence, dont
    les sommets sont représentés par les entiers de 0 à n-1.

    s0 (int) : le sommet source, par défaut le sommet 0.

    Sortie
    -------
    Renvoie la liste des prédecesseurs de chaque sommet s
    dans un plus court chemin de la source s0 à s
    '''
    n = len(G)
    dist = []
    pred = []
    a_traiter = [s0]
    vu=[False for i in range(n)] #tous les sommets sont à voir
    for s in range(n):
        dist.append(inf)
        pred.append(-1)
    dist[s0]=0 #on met la distance du depart à 0
    while a_traiter != []: #tant qu'on a des sommets a traiter
        s = a_traiter.pop(0) # supprime et prend la valeur du premier élément
        if not vu[s]:
            vu[s]=True
            for t in G[s]: #pour chaque sommet adjacent de s
                a_traiter.append(t)
                if dist[s] + cout(L[t]) < dist[t]: #si c'etait plus court de passer par s pour aller a t
                    dist[t] = dist[s] + cout(L[t]) #alors on assigne la nvlle dist de t
                    pred[t] = s #le pred de t devient s
    return pred
```

ANNEXE 9 :

```
def coord_a_indice(L,P):
    """

    Arguments
    ----------
    L : liste des sommets possibles avec les prises en coordonnées
    P : liste de la position des prises

    Sortie
    -------
    L_indices : liste des sommets possibles avec les prises en indices

    """
    L_indices = []
    for k in L:
        mg,md,pg,pd = k
        K = []
        for p in range(len(P)):
            if P[p] == mg:
                K.append(p)
        for p in range(len(P)):
            if P[p] == md:
                K.append(p)
        for p in range(len(P)):
            if P[p] == pg:
                K.append(p)
        for p in range(len(P)):
            if P[p] == pd:
                K.append(p)
        L_indices.append(K)
    return L_indices
```

ANNEXE 10 :

```
def retrace(e,dij):
    """


    Arguments
    ----------
    e : l'indice d'un état auquel on veut arriver
    dij : liste renvoyé par Dijkstra

    Sortie
    -------
    Le pcc du sommet 0 jusqu'a e

    """
    if e == 0:
        return -1
    pcc = []
    while e != 0:
        if dij[e] == -1:
            return "Trajet impossible"
        else:
            pcc.append(dij[e])
            e = dij[e]
    return pcc
```

Incertitudes :

Mesures dynamomètre

Type A :

Fc1=37,9 N

Fc2=34,1 N

Fc3=34,9 N

Moyenne : $m = 35{,}63$ N

écart-type : $$s = \sqrt{\frac{(Fc1 - m)^2 + (Fc2 - m)^2 + (Fc3 - m)^2}{(3 - 1)}}$$

$$U_A = \frac{s}{\sqrt{3}}$$

Type B : $$U_B = \frac{\text{demie-étendue}}{\sqrt{3}}$$